跟骨侧位 X 光分割与临床角度自动测量

项目技术说明

数据：跟骨侧位 X 光，11 例带标注样本 + 74 张无标注片 · 实现：PyTorch · 文档版本：2026-10

本项目实现一条从跟骨侧位 X 光片到两个临床角度（Böhler 角、Gissane 角）的自动测量链路，并在此基础上量化**「分割误差如何传导为角度误差」**——回答「继续优化分割对角度结果是否还有收益」这一方向性问题。

三项主要产出：

1. **分割**：4 层 U-Net（1,942,289 参数）从侧位片输出跟骨二值掩膜；
2. **几何测量**：从掩膜上边界轮廓定位 4 个解剖关键点，用余弦定理计算两个角度，复现误差 Böhler **0.34°**、Gissane **6.31°**；
3. **误差分析**：以三支线对照实验把角度总误差分解为「关键点提点规则误差」与「分割边界误差」，并给出 11 折留一交叉验证的结果分布与可复现性实测。

1 临床背景与任务定义

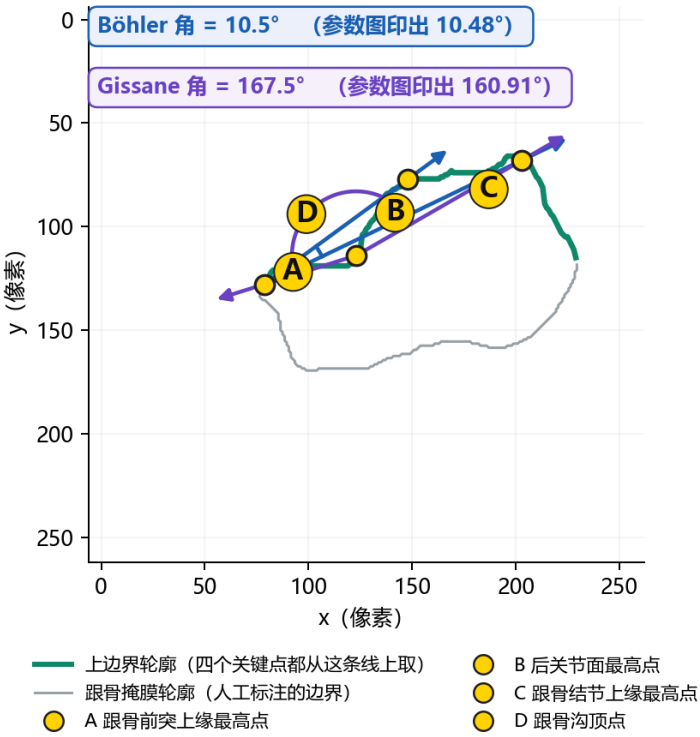
跟骨关节内骨折的评估依赖两个放射学角度。传统做法由医生在 X 光片上手动画测量线，耗时且测量者之间一致性有限。

角度	解剖含义	正常范围	骨折塌陷时
Böhler 角	跟骨后关节面与跟骨结节上缘之间的夹角	25° ~ 40°	**变小**
Gissane 角	跟骨沟处的夹角	120° ~ 145°	**变大**

本项目使用的 11 例样本实测：Böhler 3.65° ~ 18.99°、Gissane 145.46° ~ 178.48°，偏离方向与「关节内跟骨骨折塌陷」的典型表现一致。

任务定义：输入一张跟骨侧位 X 光片，输出跟骨轮廓掩膜、四个解剖关键点（A 前突上缘最高点、B 后关节面最高点、C 跟骨结节上缘最高点、D 跟骨沟顶点）以及由它们决定的两个角度；并给出该测量链路的误差量级与主要误差来源。

图 1　两个角是怎么量出来的　—　用真实数据：第 36 例的人工标注轮廓



用第 36 例的**人工标注轮廓**真实绘出。绿色曲线为上边界轮廓，四个黄点为 A / B / C / D 关键点，蓝线是以 A 为顶点的 Böhler 角两条测量臂，紫线是以 D 为顶点的 Gissane 角两条测量臂。注意 A 与 D 之间的那段轮廓接近水平——这正是后文 B 点难以自动定位的原因之一。

2 数据

数据	数量	内容	是否可用于训练
带标注样本	**11 例**	256×256，含原图 / 人工标注掩膜 / 一份参考预测掩膜 / 参数可视化图	**唯一可训练的标注数据**
原始临床片	74 张	仅原图，无任何标注	仅用于预处理验证与无监督推理

带标注样本编号：`1, 13, 23, 31, 36, 61, 73, 75, 88, 91, 94`。标注核查结果：图像与掩膜尺寸全部 256×256 且一一配对；掩膜取值仅 {0, 255}，为标准二值掩膜；前景占比 10.65% ~ 16.00%（均值 13.32%）。

> **数据与隐私**：原始影像含患者信息，不随代码仓库分发。公开内容限于代码、方法，以及不含可识别患者信息的派生结果（轮廓统计、训练曲线、对照图）。预处理脚本会将原始片重命名为 `case_001…`，并把「原名 → 编号」的映射写入单独标注的文件，该文件不纳入版本控制。

3 方法

3.1 分割网络

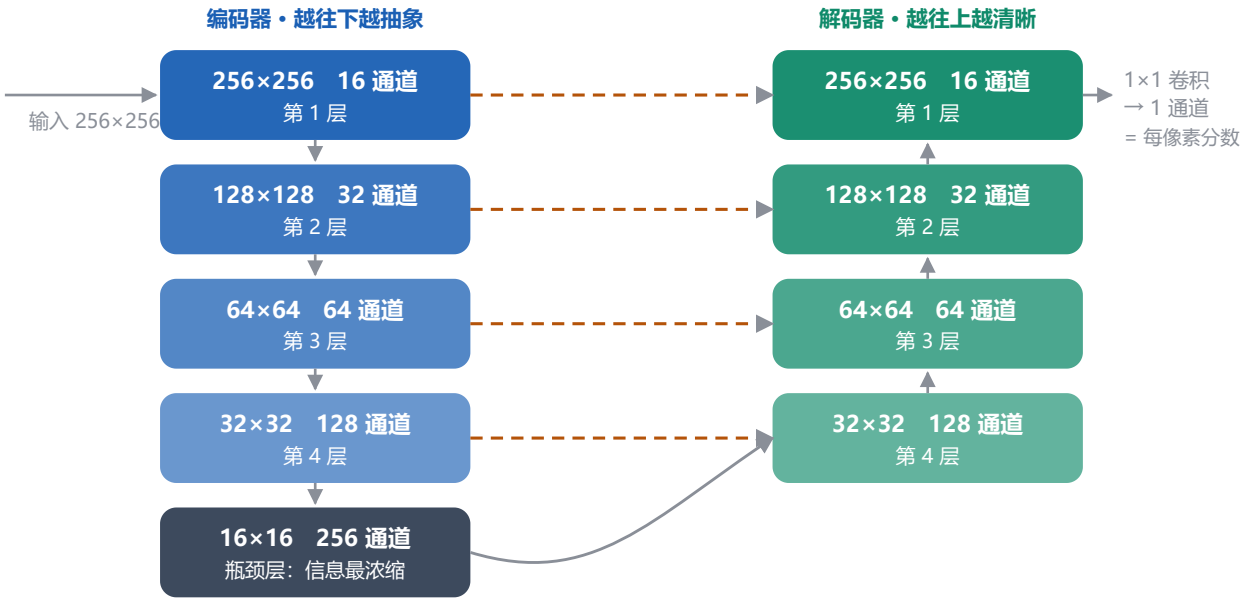
采用标准 4 层 U-Net：编码器 4 次下采样、瓶颈层、解码器 4 次上采样，每层为「两个 3×3 卷积 + BatchNorm + ReLU」。编码器每层的高分辨率特征经 skip connection 直接拼接到解码器同层，用于保住边界精度。输出为单通道 logits，预测时经 `sigmoid > 0.5` 二值化。损失函数为 `BCEWithLogitsLoss`。

层级	分辨率	通道数	说明
编码 1	256×256	16	输入层，保留最细的边界信息
编码 2	128×128	32	
编码 3	64×64	64	
编码 4	32×32	128	
瓶颈	16×16	256	信息最浓缩的一层
解码 4→1	32×32 → 256×256	128 → 16	逐级上采样，并与编码器同层特征拼接

参数量随通道基数 `base` 变化：`base=8` → 486,409；`base=16`（本项目配置）→ **1,942,289**；`base=32` → 7,762,465。

图 2　U-Net 长什么样：左边一路「下采样」变抽象，右边一路「上采样」还原细节

每个方块 = 两个 3×3 卷积 + 归一化 + 激活。分辨率越来越小、通道数越来越多，就是「越来越抽象」。



橙色虚线 = 跳连（skip connection）：把左边每一层的细节直接送到右边同一层，让轮廓边界不糊。
这是 U-Net 相对普通「编码—解码」结构的唯一关键改动，也是它做医学图像分割特别好用的原因。

3.2 几何测量（纯数学，不涉及训练）

分割之后的全部步骤都是确定性的几何运算，可独立验证：

1. **取上边界轮廓** `y_top(x)`：对掩膜的每一列 x，取最靠上的前景像素；
2. **定位四个关键点**：A 前突上缘最高点、B 后关节面最高点、C 跟骨结节上缘最高点、D 跟骨沟顶点；
3. **余弦定理求角**：以顶点出发的两条向量 u、v 计算夹角。

$$\cos\theta = (u \cdot v) / (|u| \cdot |v|)$$

$$\theta = \arccos(\cos\theta) \times 180 / \pi$$

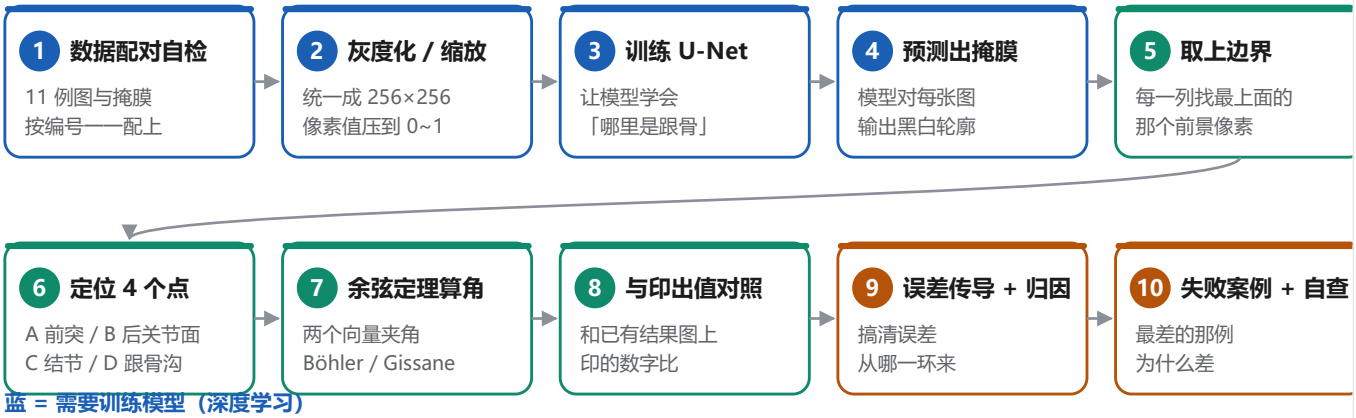
此外实现了两套与角度无关的度量，用于后续的误差分析：

- **轮廓对称表面距离**：以 numpy 实现的精确欧氏距离变换计算预测掩膜与人工掩膜之间的 mean SSD 与 HD95；
- **角度敏感度**：对每个关键点的 x / y 方向用中心差分求导，得到 `dθ/dp`（度 / 像素）。

本项目不使用 scipy 与 OpenCV，距离变换与形态学操作均由 numpy 自行实现，依赖面收敛到 `torch / numpy / Pillow / matplotlib` 四个包。

图 3　全流程十步：前 4 步要训练模型，中间 4 步是纯数学，最后 2 步是分析

记住这条分界线很重要 —— 被问到「哪部分是深度学习、哪部分是传统方法」时，答案就是第 4 步与第 5 步之间。



3.3 角度口径

本链路采用的顶点取法与骨科教科书定义不同，这是为对齐已有参考结果而做的选择：

角度	本链路口径	教科书口径
Böhler	顶点 = A（前突点）	顶点 = B（后关节面点）
Gissane	顶点 = D，两臂到 A 与 C	顶点 = D，两臂到 A 与 B

若按教科书定义计算，Böhler 角会得到 118° ~ 168°，与参考结果上印出的 3.65° ~ 19° 完全对不上。因此**复现既有结果时，第一步应对齐口径，而不是对齐代码**；这一结论已用 11 例逐例验证（见 4.2 节）。

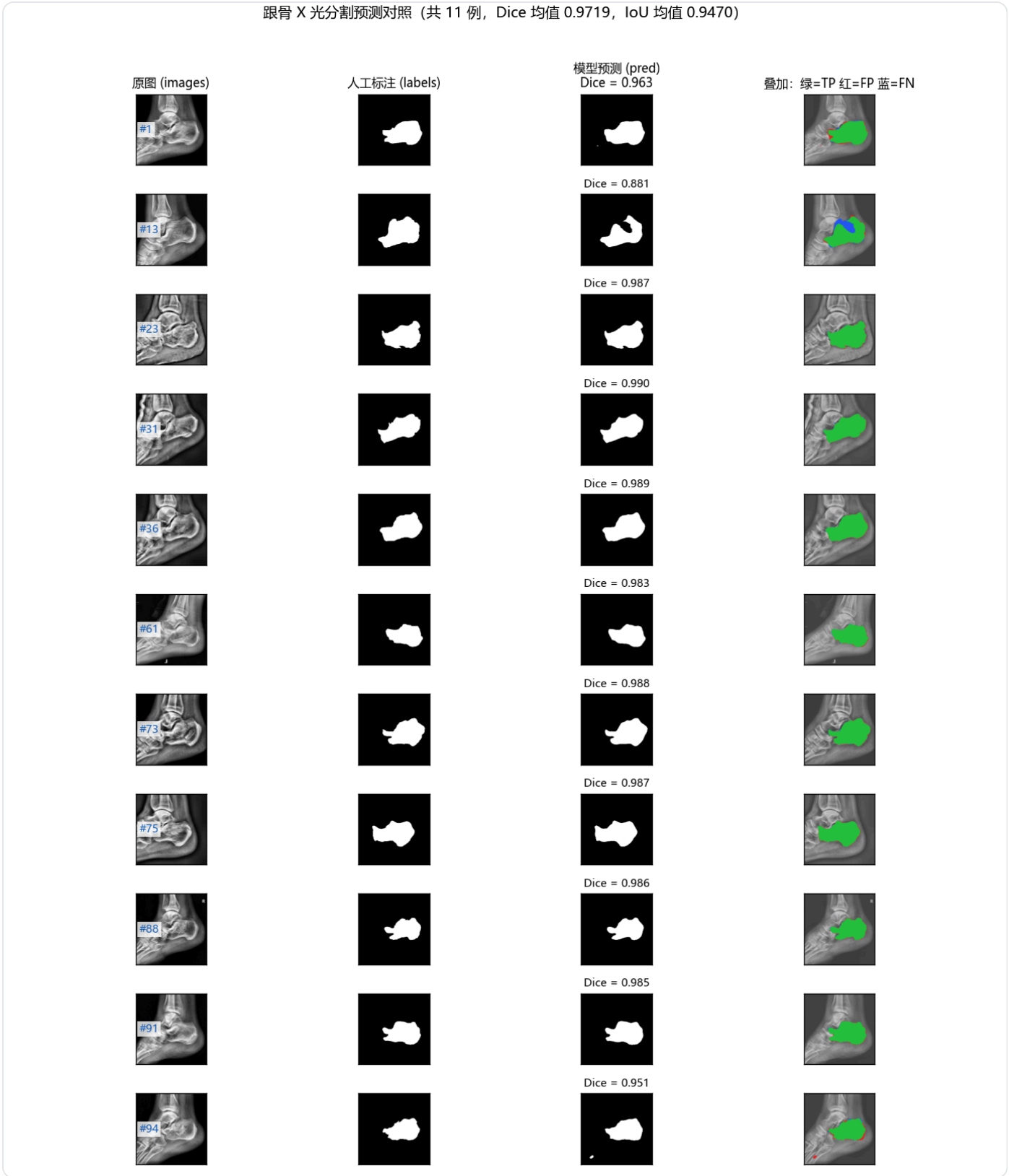
4 实验结果

4.1 分割

实验	训练 Dice	验证 Dice	验证 IoU	耗时
过拟合测试（11 例全部训练）	0.9920	—	—	10.1 s
8 训练 / 3 验证（seed = 42）	0.9933	**0.9318**	0.8574	10.7 s

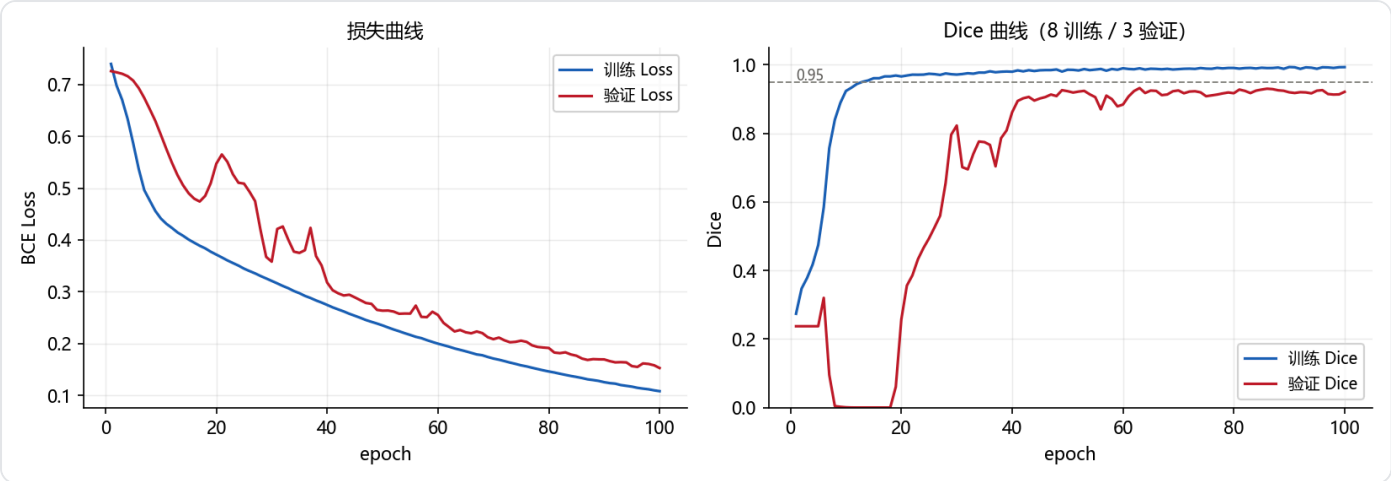
用留出权重预测全部 11 例，可以清楚区分「记住的」与「没见过的」：

类型	样本	Dice
验证集（未见）	1 / 13 / 94	0.9631 / **0.8814** / 0.9510
训练集（见过）	23, 31, 36, 61, 73, 75, 88, 91	0.983 ~ 0.990



仓库产物 `results/calcaneus_predict_grid_split83.png`：全部 11 例的预测对照。第 4 列是三色叠加——**绿 = 预测正确、红 = 多预测、蓝 = 漏预测**。其中第 13 例的蓝色区域明显异常，即 4.7 节专门分析的失败案例。

训练过程中第 10 ~ 15 轮验证 Dice 一度塌到 0（模型退化为「全预测背景」），第 30 轮后自行恢复。这是分割训练中的典型 collapse 现象，已如实保留在实验记录中，并在 4.5 节用留一验证进一步定位。



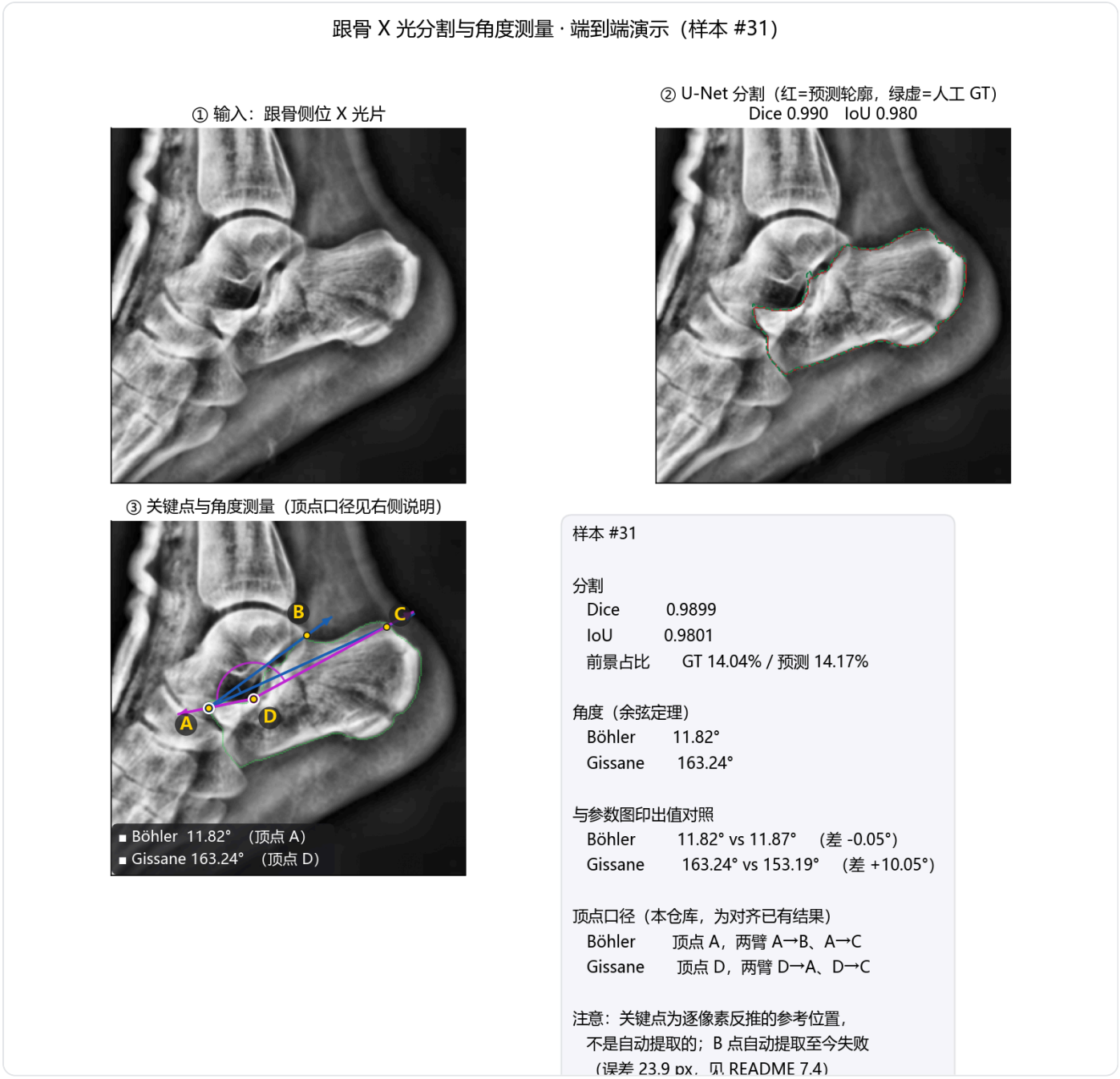
仓库产物 `results/calcaneus_train_curve_split83.png`。训练曲线：蓝线为训练集 Dice（持续上升），红线为验证集 Dice——它在第 10 ~ 15 轮跌到 0，第 30 轮后自行恢复，即 4.1 节所述的 collapse 现象。

4.2 角度复现

用参考关键点跑完整条测量链路，与参考结果图上印出的数值逐例对照：

角度	11 例平均绝对误差	最大误差
Böhler	**0.34°**	1.01°
Gissane	6.31°	12.36°

Böhler 的复现误差在 0.34° 量级，说明端点定义已被准确还原；Gissane 的残差明显更大，原因是它的顶点 D 位于形态复杂、曲率变化剧烈的跟骨沟区域，而关键点位置来自对参考图的逐像素反推，该区域的反推精度本身就受限。这一残差在后续的误差归因中被作为「基线误差」单独处理，不计入分割误差。



仓库产物 `results/calcaneus_demo_case31.png`，由 `python demo_single_case.py` 生成。端到端单例演示：左上原片、右上分割结果（红 = 模型轮廓，绿虚 = 人工轮廓）、左下关键点与测量臂、右下数值卡。

4.3 误差传导

先测分割边界误差，再测角度对每个关键点的敏感度，两者相乘，得到角度误差的量级边界：

指标	结果
分割边界平均偏差（mean SSD）	0.91 ~ 3.05 px，均值 **1.56 px**
HD95	2.83 ~ 16.12 px
Böhler 角最敏感的点	**B（0.74 °/px）**
Gissane 角最敏感的点	**D（2.28 °/px）**
传导出的角度误差（局部折算）	Böhler ±2.79°，Gissane ±2.10°
传导出的角度误差（保守估计）	Böhler ±1.45°，Gissane ±4.45°

> **结论一：分割边界平均只偏 1.56 像素，就足以让角度抖动 ±2 ~ 4 度。**
>
> 这意味着在当前分割精度下，把角度测到小数点后两位没有实际意义。

需要说明本节所用掩膜的归属：上述 1.56 px 指的是**与数据一同提供的参考预测掩膜**的边界误差，不是本项目自训 U-Net 的输出。两者的质量对照见 4.7 节。

4.4 误差归因：总误差到底来自哪一环

4.3 节的结论里混着两个误差源：**关键点提点规则本身不准**，和**掩膜边界误差传导到关键点位置**。不拆开就无法回答「下一步该优化哪里」。为此设计了 3 条支线、2 个减法的因子对照：

支线	组合	减法含义
①	人工掩膜 + 参考关键点 → **A1**	基线
②	人工掩膜 + 自动提点 → **A2**	**A2 − A1 = 提点规则误差**（掩膜不变，只换提点方式）
③	预测掩膜 + 自动提点 → **A3**	**A3 − A2 = 分割边界误差**（提点规则不变，只换掩膜）

> 原计划中的第 4 条支线「预测掩膜 + 参考关键点」是退化的：角度只由关键点坐标决定，关键点一旦固定在参考位置，掩膜便不再影响结果，该格必然恒等于 A1。因此正确的因子设计只能到 3 格。

结果（11 例平均绝对误差，单位：度）

角度	基线复现	提点规则 (A2−A1)	分割边界 (A3−A2)	端到端 (A3 vs 印出值)
Böhler	0.34	**6.19（中位 4.27）**	4.90（中位 1.41）	7.50
Gissane	6.31	**13.37（中位 8.07）**	6.17（中位 2.43）	11.08

结论二：两个角的最大误差源都是「提点规则」，不是分割边界。

Böhler：6.19° vs 4.90°（均值 1.3 倍，**按中位数看是 3.0 倍**）；Gissane：13.37° vs 6.17°（均值 2.2 倍，**按中位数看是 3.3 倍**）。

⇒ **继续提升分割精度对角度结果没有收益**，唯一有效的方向是获得关键点的准确定义与标注方式。这一条把一个方向性问题回答掉了。

与 4.3 节的交叉印证：4.3 节用「敏感度 × 边界偏移」独立折算得到 ±1.45 ~ 2.79°（Böhler）/ ±2.10 ~ 4.45°（Gissane）；本次通过直接重提点测到的「分割边界」贡献中位数是 **1.41° / 2.43°**——两种独立方法给出同一量级。均值（4.90° / 6.17°）被个别大误差样本抬高。

三条需一并说明的边界：

① A1 基线自身带误差，分解是在「以反推参考点为真值」的前提下成立的。Gissane 的基线误差（6.31°）与其分割贡献（6.17°）同量级，因此 **Gissane 的归因结论弱于 Böhler**。

② 端到端误差（7.50° / 11.08°）同时包含上述三项，其中「基线复现」一项混有参考点反推误差，不能全部计入该链路的误差。

③ 以上均为 11 例上的结果，不是统计结论。

图 7　误差是怎么从掩膜传到角度上的，以及它到底该怪谁

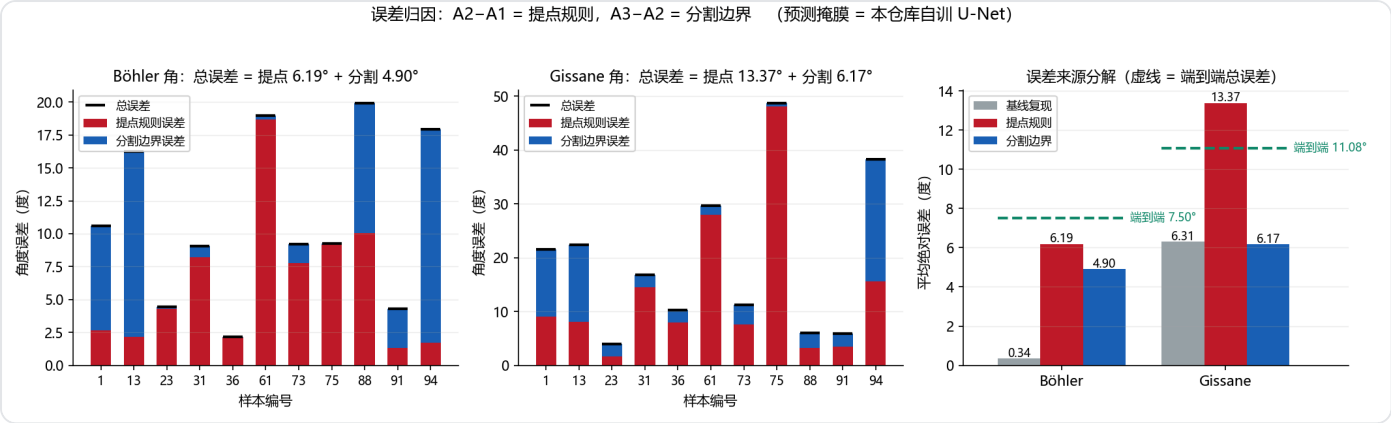
上半条是因果链：掩膜偏一点，关键点就偏一点，角度就跟着抖。下半条是归因结论：抖出来的误差，主要怪提点规则，不怪分割。



但这个总误差可以拆成两半：



结论：主要误差在「提点规则」那一侧 ⇒ 继续把分割做得更准，对角度结果没有帮助。



仓库产物 `results/calcaneus_error_attribution.png`，由 `python error_attribution.py` 生成。左侧为三个角度的逐例误差，右侧为误差来源分解：灰 = 基线复现、红 = 提点规则、蓝 = 分割边界，绿色虚线为端到端总误差。可以直观看到两个角的最大来源都是「提点规则」。

4.5 留一交叉验证与训练稳定性

原来的 8/3 划分只用 3 例作为测试，Dice 在 0.88 ~ 0.96 之间波动，无论说好说坏都没有依据。改用 11 折留一交叉验证，让**每一例都当过一次测试样本**，从而得到分布而非三个数字。

配置	Dice 均值 ± 标准差	中位数	四分位区间	最小值
`BCE`（原配置）	0.8327 ± **0.2414**	0.9406	0.9012 – 0.9526	**0.3333**
`BCE + Dice`	0.9499 ± **0.0253**	0.9536	0.9449 – 0.9664	0.8827

发现：11 折中有 2 折完全崩溃。第 31 例 Dice 0.3598、第 73 例 Dice 0.3333；两折的预测前景占比分别冲到 63.8% 与 68.2%（真值仅约 14%），即模型退化为「全前景」，面积比 4.5 / 4.7。这与 4.1 节 8/3 训练中观察到的「验证 Dice 塌到 0」（退化为全背景）是同一类现象的两个方向。

崩溃是「种子依赖」而非「样本依赖」——对这两折更换随机种子重跑：

测试例	seed = 42	seed = 1042	seed = 2042
第 31 例	**0.3598**	0.8343	**0.9635**
第 73 例	**0.3333**	**0.9505**	**0.9418**

同一例更换种子可从 0.33 变为 0.96，同折不同种子的 Dice 极差最大达 0.6172。说明崩溃是训练不稳定的一次随机落点，而不是「这两例特别难」。

干预：同种子、同划分，唯一变量为损失函数。

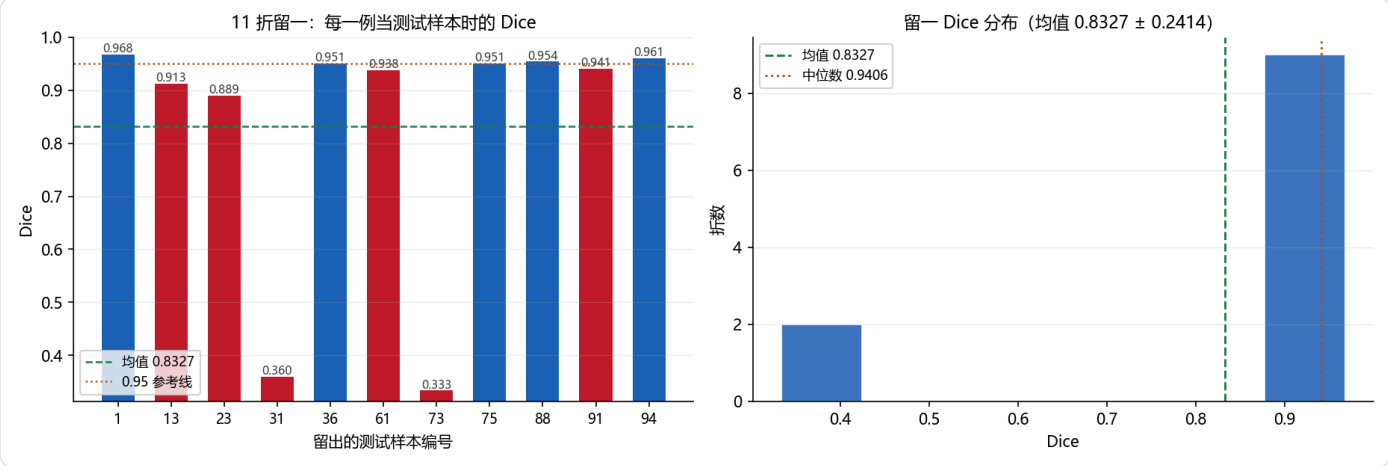
配置（seed = 42）	第 31 例	第 73 例	面积比
`BCE`	0.3598	0.3333	4.54 / 4.75
`BCE + Dice` （0.5×BCE + 0.5×soft-Dice）	**0.8827**	**0.9536**	1.15 / 1.07

跑满 11 折后标准差由 0.2414 降至 0.0253（缩小约 10 倍），最小值由 0.3333 抬升到 0.8827。

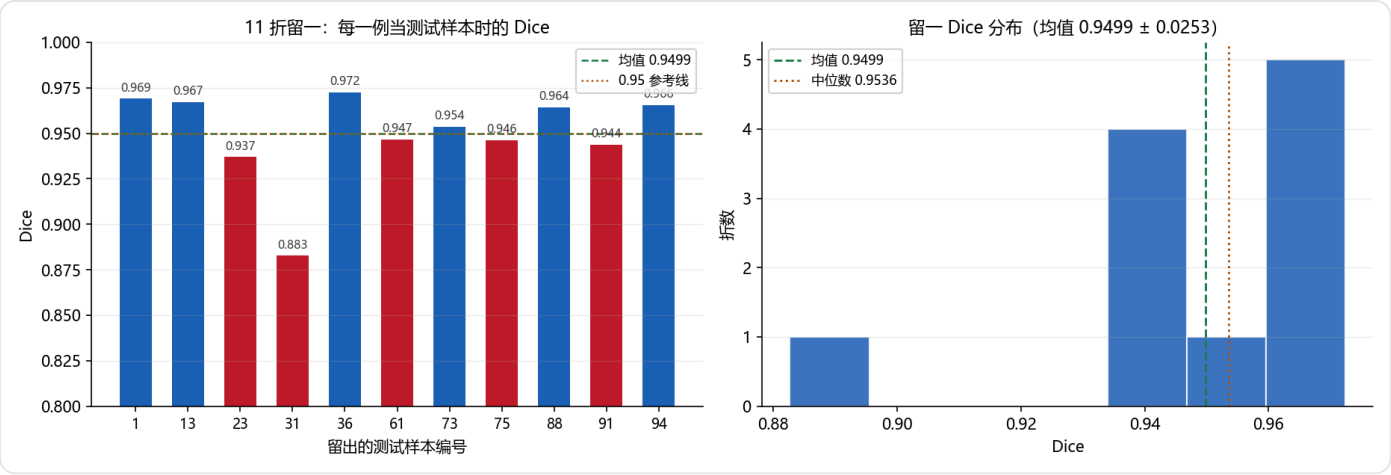
> 准确表述是「**混合损失降低了训练崩溃的概率，使结果不再对随机种子敏感**」，而不是「混合损失更准」——对照实验中 `BCE` 更换种子同样可以跑到 0.94 ~ 0.96。

顺带推翻 8/3 划分的那次结论：验证集是 1、13、94，这三例在留一中的成绩恰好是 0.9677 / 0.9129 / 0.9611——那次划分运气较好，未抽到崩溃折。

> 留一给出的是 11 例上的结果分布，**仍然不是泛化性能**。它比 8/3 划分强的地方在于消除了「哪 3 例当验证集」这一运气因素，但样本量小这一根本问题没有被解决，只是被量化了。另外留一每折用 10 例训练（8/3 划分只用 8 例），训练集更大，两者不能直接比大小。



仓库产物 `results/calcaneus_loo.png`。左图为每一例作为留出测试样本时的 Dice，右图为分布。两根红色柱（第 31 例 0.3598、第 73 例 0.3333）即训练崩溃的两折。



更换为 `BCE + Dice` 混合损失后重跑同一套 11 折留一：崩溃折消失，分布明显收紧。这张图与上一张为**同一个种子、同一份划分，唯一变量是损失函数。**

4.6 可复现性

固定随机种子只能固定数据划分与参数初始化，**固定不了 GPU 上的非确定性**——cuDNN 会自动挑选卷积算法，同一份数据两次运行可能落到不同实现上，误差逐轮累积。本项目对此做了实测（同一台机器、同一份数据、同一份代码、同一 `seed=42`）：

运行配置	8-3 最好验证 Dice	第 13 例 Dice	11 例 Dice 均值
未开确定性卷积，运行 A	0.9318	0.8814	0.9719
未开确定性卷积，运行 B	0.9427	—	—
未开确定性卷积，运行 C	0.9455	—	—
开启 `cudnn.deterministic`，运行 D	**0.9461**	**0.9216**	**0.9785**
开启 `cudnn.deterministic`，运行 E	**0.9461**	—	—

- 关闭确定性时：两次运行的逐轮指标**从第 1 轮起就不同**，最好验证 Dice 相差 0.3 个百分点；
- 开启确定性后：两次运行**逐轮 bit 级完全一致**（0.9461320241292318），结果可复现。

> **结论三：1.4 个百分点的运行漂移，大于任何模型改动能带来的提升。**
>
> 在 11 例 / 3 例验证集这个规模上，讨论「模型 Dice 是 0.93 还是 0.94」没有意义——这正是本项目不提供泛化性能数字的量化依据。

工程上，训练入口的种子函数已默认开启 `cudnn.deterministic` 并关闭 `cudnn.benchmark`；`run_all.py` 同时固定 `PYTHONHASHSEED`。需要注意：仓库 `results/` 中记录的是开启确定性之前的一次运行，因此完整重跑所得数字与已记录

数字**不逐位相同**——数值敏感的产物（训练日志、训练曲线、逐例 Dice）会有小幅差异，而**不依赖模型的产物**（`calcaneus_angles_*.csv/png`，只依赖掩膜与参考关键点）重跑完全一致。这一点已在文档中说明，属于有意保留的实验记录。

4.7 失败案例与掩膜质量对照

第 13 例是本库唯一一例明显失败：Dice 0.8814（其余 10 例均值 0.9809），IoU 0.7880（其余 0.967+），属于两个水平的差距而非「稍微差一点」。逐项指标：

指标	第 13 例	其余 10 例
漏检像素 FN	**2022 px**	13 ~ 121 px
误检像素 FP	254 px	134 ~ 609 px
上边界平均压低	**6.92 px**	0.53 ~ 1.92 px
HD95	**30.60 px**	1.00 ~ 7.07 px
真值前景占比	16.00%（全库最高）	10.65 ~ 14.93%
预测前景 / 真值前景	13.30% / 16.00% = **0.83 倍**	约 1.0 倍

失败方向明确：漏检 2022 px 对误检 254 px，比例 8:1——模型不是「多画了一块」，而是**整块地少画了一块**，预测前景只有真值的 0.83 倍，轮廓被整体「缩」了。

三条假设用数据判定，其中一条被排除：

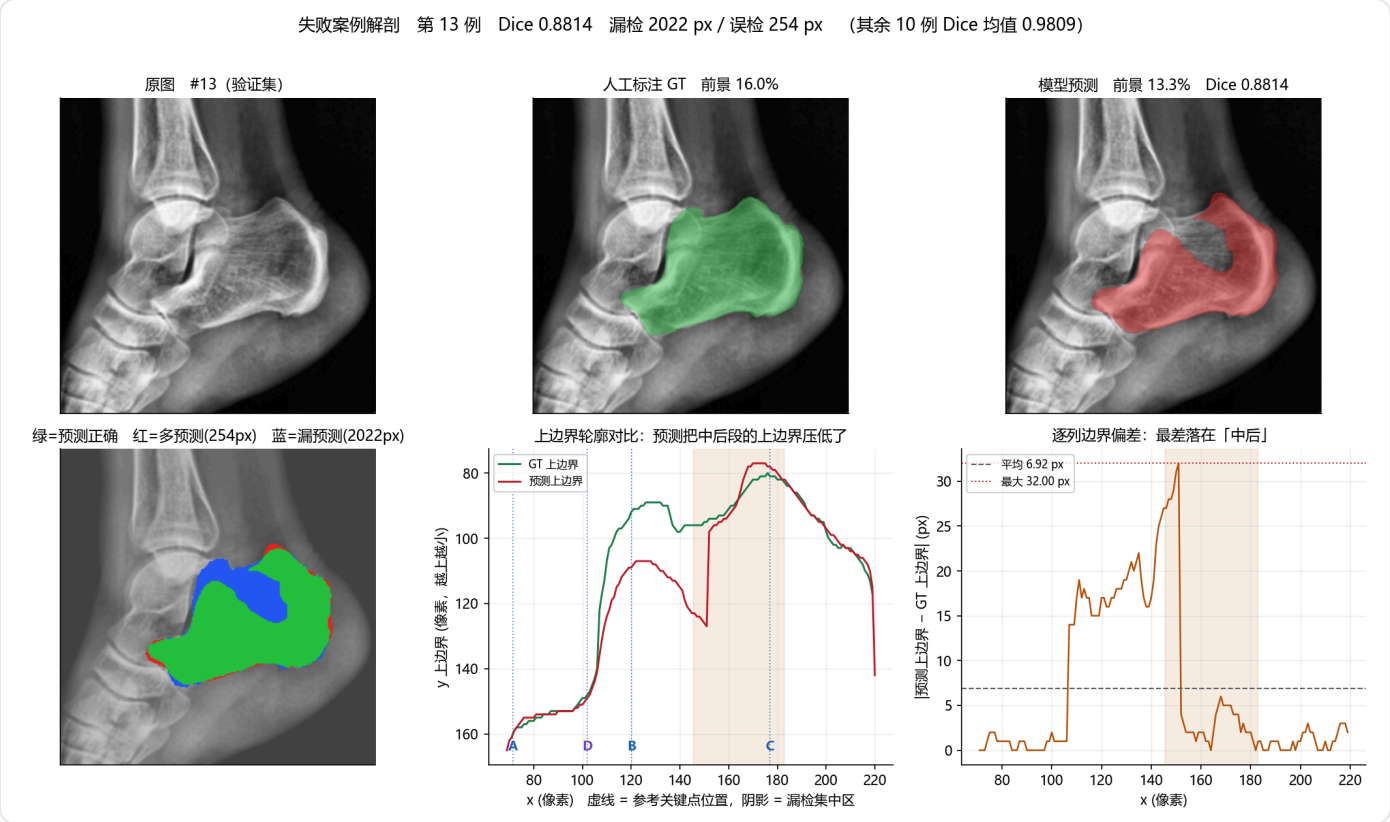
假设	证据	判定
该例曝光 / 对比度异常，边界本身不清晰	灰度对比度（p99−p1）231.0 vs 其余均值 229.7，比值 1.01	**不支持**
目标面积与训练分布不匹配，模型整体「缩」了	预测前景 13.30% ÷ 真值前景 16.00% = 0.83 倍	**支持**
漏检集中在特定区域，该处边界形态与训练样本差异大	中后段漏检密度 0.1250，是四段均值 0.0530 的 2.4 倍	**支持**

按 x 方向（前突侧 → 结节侧）四等分统计错误像素密度，漏检集中在中后段（0.1250），位置落在 B 点（x≈120）与 C 点（x≈177）之间，即后关节面到跟骨结节之间的那段上缘；逐列偏差在该处冲到最大 32 px，其余位置基本在 5 px 以内。这条**位置信息**比「Dice 掉了多少」更有指导意义——它直接指出应针对哪里做定向数据增强或加权损失。

与参考掩膜的质量对照（同为与真值比较）：

	合并 IoU	合并 Dice	第 13 例 IoU
随数据提供的参考预测掩膜	0.9275	0.9624	**0.9274**
本项目自训 U-Net	**0.9445**	**0.9715**	**0.7880**

两条结论需要同时说明：**整体上自训模型更好**（IoU +0.017），但**在第 13 例上明显更差。**结合 4.6 节，+0.017 的 IoU 优势与运行噪声同量级，因此不宜表述为「本方法优于该参考结果」，只宜表述为「两者水平相当，且各有表现更差的案例」。



仓库产物 `results/calcaneus_case13_analysis.png`，由 `python analyze_failure_case.py` 生成。6 格依次为：原图 / 人工标注 / 模型预测 / 三色叠加 / 上边界曲线对比 / 逐列边界偏差。预测掩膜在「中后段」被整块削去（蓝色漏检区），偏差峰值落在 B 点与 C 点之间。

5 已知局限

1. **样本量**：全库仅 11 例带标注，验证集仅 3 例，Dice 在 0.88 ~ 0.96 之间波动，**因此本项目不提供任何泛化性能结论。**过拟合测试的 0.9920 仅用于验证技术链路连通，不代表模型效果。
2. **关键点定义来自反推**：角度公式已复现（Böhler 误差 0.34°），但 B 点的定位规则未能从轮廓几何还原——无论采用「曲率最大处」「陡升终点」还是「后关节面段最高点」等规则，与参考位置的偏差都在 23.9 px 量级，说明其定义不完全由轮廓几何决定。当前 D 点可可靠自动提取（3.9 px），A / C 点勉强（6.9 / 10.2 px）。因此这条链路是「差一环的全自动」：分割与几何计算均已自动化，关键点仍依赖参考位置。
3. **归一化方式**：当前使用逐图 min-max，会掩盖不同片子的曝光差异；更稳妥的做法是固定窗宽窗位或分位数截断。
4. **未做部署**：模型尚未导出 ONNX，也未做推理延迟与显存测量。

6 复现方式

环境：Python 3.10 + PyTorch 2.11.0（cu128）+ CUDA；依赖 numpy、Pillow、matplotlib，版本见 `requirements.txt`。不使用 scipy 与 OpenCV。参考测试环境为单卡 8 GB 显存的消费级 GPU；无 GPU 亦可运行，仅速度较慢。

一条命令跑完整条链路（推荐）

```
pip install -r requirements.txt
python run_all.py                 # 全部步骤，约 80 秒
python run_all.py --tag check  # 产物名加后缀，不覆盖已记录结果
```

`run_all.py` 会固定随机种子与 `PYTHONHASHSEED`，按正确顺序调用各步骤，最后打印逐步耗时与成败汇总。

分步运行

```
python calcaneus_dataset.py            # 数据配对与张量形状检查
python unet.py                         # 网络结构与三种规模参数量
python train_calcaneus.py --overfit --epochs 80 --tag overfit
python train_calcaneus.py --epochs 100 --tag split83
python predict_calcaneus.py --weights results/calcaneus_unet_best_split83.pth
--tag _split83
python calcaneus_angles.py --mask results   # 角度复现 + 敏感度 + 误差传导
python analyze_failure_case.py         # 失败案例解剖
python leave_one_out.py                 # 11 折留一，约 2 分钟
python error_attribution.py            # 误差归因
python demo_single_case.py             # 端到端单例演示
```

数据根目录通过候选路径自动探测，也可用环境变量 `CALCANEUS_ROOT` / `CALCANEUS_RAW` 覆盖。数据未就位时入口脚本会给出明确提示，不会静默失败。

7 代码结构

文件	职责
`calcaneus_dataset.py`	数据加载：按文件名编号配对、灰度化、缩放、归一化、二值化
`unet.py`	U-Net 模型定义（DoubleConv + UNet）
`train_calcaneus.py`	训练与验证循环，固定随机种子，按最优验证 Dice 保存权重
`predict_calcaneus.py`	加载权重预测并生成四列对照图（含 TP/FP/FN 三色叠加）
`calcaneus_angles.py`	角度公式、自实现欧氏距离变换、中心差分敏感度、误差传导、关键点自动提取
`analyze_failure_case.py`	失败案例解剖：逐例指标表、6 格解剖图、三条归因假设的数据化判定
`leave_one_out.py`	11 折留一交叉验证，支持单折诊断、多随机种子、损失函数对比
`error_attribution.py`	误差归因：三支线因子设计，分解提点规则误差与分割边界误差
`demo_single_case.py`	端到端单例演示：分割 → 轮廓 → 关键点 → 角度，输出 4 格演示图与数值卡
`run_all.py`	一键复现入口：按序调用全部步骤，固定种子与 hash seed，汇总耗时与成败
`calcaneus_preprocess.py`	原始片脱敏重命名与尺寸、灰度统计

8 参考

- Ronneberger O, Fischer P, Brox T. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. MICCAI 2015.

本文档中所有数值均由仓库内脚本在本机实测得到，可逐项对应到 `results/` 下的结果文件与对应脚本。

原始影像含患者信息，不随代码仓库分发；本文档不含任何可识别患者信息的内容。